

Swift integration

supabase-swift for Onboard

SETUP

Install & client

Codable models

AUTH

Sign in / up / restore

DATA

Fetching (nested selects)

Insert, update, delete

RPC (employee portal)

Storage uploads

PATTERNS

Error handling

Realtime

Swift integration guide

How to wire a Swift/SwiftUI client to the same Supabase project the web app uses, with `supabase-swift`. Models and calls below map 1:1 onto the tables and RPC functions in the [API reference](#) — same project, same rows, same rules.

Install & client setup

Add the package via Swift Package Manager:

Package.swift

```
dependencies: [  
  .package(url: "https://github.com/supabase/supabase-swift.git", from: "2.0.0")  
]
```

Create one shared client. Get the URL and anon key from the web team — same values as their `.env`.

SupabaseClient.swift

```
import Supabase  
  
let supabase = SupabaseClient(  
  supabaseURL: URL(string: "https://your-project-ref.supabase.co")!,  
  supabaseKey: "your-anon-key"  
)
```

Don't hardcode these two values in source you'll commit — pull them from an `.xcconfig` or the Keychain, same instinct as the `.env` file on web.

Codable models

Match the Postgres column names with `CodingKeys` — the tables use `snake_case`, Swift wants `camelCase`.

Models.swift

```

struct Company: Codable, Identifiable {
    let id: UUID
    let name: String
    let industry: String?
    let departments: [String]
    let primaryColor: String?
    let logoUrl: String?

    enum CodingKeys: String, CodingKey {
        case id, name, industry, departments
        case primaryColor = "primary_color"
        case logoUrl = "logo_url"
    }
}

struct Employee: Codable, Identifiable {
    let id: UUID
    let name: String
    let email: String
    let department: String?
    let employmentType: String?
    let assignedProgramId: UUID?
    let accessToken: UUID

    enum CodingKeys: String, CodingKey {
        case id, name, email, department
        case employmentType = "employment_type"
        case assignedProgramId = "assigned_program_id"
        case accessToken = "access_token"
    }
}

struct Program: Codable, Identifiable {
    let id: UUID
    let name: String
    let status: String
    var stages: [Stage]? // nil unless you embedded it in the select
}

struct Stage: Codable, Identifiable {
    let id: UUID
    let name: String
    let order: Int
    let deadline: Int?
    var materials: [Material]?
}

```

`materials.data` is jsonb — its shape depends on `type`. Don't try to model every variant as one struct; decode `data` as a raw `JSONObject` and switch on `type` where you render it.

```
struct Material: Codable, Identifiable {
    let id: UUID
    let type: String // "video" | "reading" | "checklist" | "quiz" | "task" | "document" |
    let title: String
    let data: JSONObject // from Supabase's own AnyJSON / JSONObject type
}
```

Gotcha: `data` holds different keys per type (`url` / `content` for reading, `questions` for quiz, `items` for checklist...). Read the field list in the [API reference](#)'s data model section before building your material views.

Sign in, sign up, restore session

AuthManager.swift

```
func signIn(email: String, password: String) async throws {
    try await supabase.auth.signIn(email: email, password: password)
    // session is persisted automatically – no manual token storage needed
}

func restoreSession() async -> Session? {
    try? await supabase.auth.session
}

func signOut() async throws {
    try await supabase.auth.signOut()
}
```

Call `restoreSession()` at app launch, before deciding whether to show the login screen — mirrors what the web app's `initSession()` does. Then fetch the profile to get role + company:

```
struct Profile: Codable {
    let id: UUID
    let role: String // "admin" | "owner"
    let companyId: UUID?

    enum CodingKeys: String, CodingKey {
        case id, role
        case companyId = "company_id"
    }
}

let profile: Profile = try await supabase
    .from("profiles")
    .select()
    .eq("id", value: userId)
    .single()
    .execute()
    .value
```

Fetching data (nested selects)

Embed related tables in one query with PostgREST's nested select syntax — same trick the web app uses to fetch a program with all its stages and materials in one round trip:

```
let programs: [Program] = try await supabase
  .from("programs")
  .select("*, stages(*, materials(*))")
  .eq("company_id", value: companyId)
  .order("created_at", ascending: true)
  .execute()
  .value
```

Stages come back in whatever order Postgres feels like — sort by `stage.order` client-side after decoding, same as the web store does.

Insert, update, delete

```
// Insert – Encodable struct, get the created row back
struct NewEmployee: Encodable {
    let companyId: UUID
    let name: String
    let email: String

    enum CodingKeys: String, CodingKey {
        case companyId = "company_id"
        case name, email
    }
}

let created: Employee = try await supabase
    .from("employees")
    .insert(NewEmployee(companyId: companyId, name: "Sari Wulandari", email: "sari@..."))
    .select()
    .single()
    .execute()
    .value

// Update – partial dictionary payload works fine too
try await supabase
    .from("employees")
    .update(["assigned_program_id": programId.uuidString])
    .eq("id", value: employeeId)
    .execute()

// Delete
try await supabase
    .from("employees")
    .delete()
    .eq("id", value: employeeId)
    .execute()
```

Always await the call. The web team shipped a bug early on from constructing a query and never awaiting it — the builder is lazy and the request simply never fires until you do. Swift's `async / await` makes this harder to get wrong, but it's the one thing worth double-checking in any fire-and-forget helper you write.

RPC — employee portal (no auth)

If the desktop app has a kiosk or "shared computer" mode where an employee opens their onboarding without signing in, call the same public RPC functions the web portal uses — pass the token, get back one JSON blob:

```
struct TokenParams: Encodable { let pToken: UUID
    enum CodingKeys: String, CodingKey { case pToken = "p_token" }
}

struct PortalResponse: Decodable {
    let employee: Employee
    let company: Company
    let program: Program?
}

let portal: PortalResponse = try await supabase
    .rpc("get_employee_portal", params: TokenParams(pToken: token))
    .execute()
    .value

// marking progress:
struct CompleteParams: Encodable {
    let pToken: UUID
    let pMaterialId: UUID
    enum CodingKeys: String, CodingKey { case pToken = "p_token", pMaterialId = "p_material_id"
}

try await supabase
    .rpc("mark_material_complete", params: CompleteParams(pToken: token, pMaterialId: materialId))
    .execute()
```

Storage — logo & banner uploads

```
let path = `\"(companyId)/logo.png\"
try await supabase.storage
  .from(\"company-logos\")
  .upload(path, data: imageData, options: FileOptions(upsert: true))

let publicURL = try supabase.storage
  .from(\"company-logos\")
  .getPublicURL(path: path)
```

The upload path's first segment must be your own `company_id` — the storage policy on the bucket checks that prefix and rejects anything else.

Error handling

Wrap calls in `do/catch` — auth errors and Postgres/RLS errors both surface as thrown errors, not optional results:

```
do {
  try await supabase.auth.signIn(email: email, password: password)
} catch let error as AuthError {
  // e.g. wrong password, unconfirmed email
  showError(error.localizedDescription)
} catch {
  showError(\"Something went wrong – try again.\")
}
```

A silent empty result (no thrown error, zero rows) almost always means RLS blocked the row rather than it not existing — check you're signed in as a user whose `company_id` matches the row before assuming the data is missing.

Realtime (optional)

Useful for a kiosk-style dashboard that should update live as employees complete steps, without polling:

```
let channel = supabase.channel("employee-progress")
let changes = channel.postgresChange(
  InsertAction.self,
  schema: "public",
  table: "employee_progress"
)
await channel.subscribe()

for await change in changes {
  // refresh the affected employee's progress
}
```